ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. ОСНОВЫ АЛГОРИТМИЗАЦИИ, ЧАСТЬ 1

1.1. Запись алгоритмов с помощью языка блок-схем

При схемном описании алгоритм изображается отдельными геометрическими фигурами (символами), связанными между собой с помощью линий потоков с однозначно заданным направлением со стрелками [22]. Внутри символов записывается выполняемая ими последовательность действий.

Данный способ по сравнению с другими способами записи алгоритма имеет ряд преимуществ. Он наиболее нагляден: каждая операция вычислительного процесса изображается отдельной геометрической фигурой. Кроме того, графическое изображение алгоритма наглядно показывает разветвления путей решения задачи в зависимости от различных условий, повторение отдельных этапов вычислительного процесса и другие детали.

Схемой алгоритма (блок-схемой) называют его графическое представление последовательность связанных между собой функциональных блоков, каждый из которых соответствует выполнению одного или нескольких действий.

В схеме алгоритмов каждому типу действий (вводу исходных данных, вычислению значений выражений, проверке условий, управлению повторением действий, окончанию обработки и т. п.) соответствует геометрическая фигура, представленная в виде символа. Символы соединяются линиями потоков данных или управления, определяющими очередность выполнения действий.

Правила выполнения схем алгоритмов устанавливаются в действующем ГОСТ 19.701-90 Схемы алгоритмов, программ данных и систем. Они строятся из имеющих заданное значение символов, краткого пояснительного текста и соединительных линий. Схемы алгоритмов могут использоваться на разных уровнях детализации, причем число уровней зависит от размеров и сложности задачи обработки данных.

В ГОСТ 19.701-90 определены символы, предназначенные для использования в документации по обработке данных, и приведено руководство по условным обозначениям для применения их в следующих случаях:

- схемах данных;
- схемах программ;

схемах работы системы;

- схемах взаимодействия программ;
- схемах ресурсов системы.

В стандарте используются следующие понятия:

1. основной символ – символ, используемый в тех случаях, когда точный тип (вид) процесса или носителя данных не известен или отсутствует необходимость в описании фактического носителя;

2. специфический символ – символ, используемый в тех случаях, когда известен точный тип (вид) процесса или носителя данных или когда необходимо описать фактический носитель данных;

3. схема – графическое представление определения, анализа или метода решения задачи, в котором используются символы для отображения операций, данных, потока, оборудования и т. д.

Схема данных отображает путь данных при решении задач и определяет этапы обработки, а также различные применяемые носители данных. Она состоит из символов данных; символов процесса, который следует осуществить над данными; символов линий, отображающих потоки данных между процессами; (или) носителями данных и специальных символов, используемых для облегчения написания и чтения схемы.

Схема программы отображает последовательность операций в программе. Она состоит из символов процесса, указывающих фактические операции обработки данных; линейных символов, отображающих поток управления; специальных символов, используемых для облегчения написания и чтения схемы.

Схема работы системы отображает управление операциями и поток данных в системе. Она состоит из символов данных, указывающих на наличие данных; символов процесса, указывающих операции, которые следует выполнить над данными; линейных символов, отображающих потоки данных между процессами и (или) носителями данных. а также поток управления между процессами; специальных символов, используемых для облегчения написания и чтения схемы.

Схема взаимодействия программ отображает путь активаций программ и взаимодействий с соответствующими данными. Каждая программа в схеме взаимодействия программ показывается только один раз (в схеме работы системы программа может изображаться более чем в одном потоке управления). Она состоит из символов данных, указывающих на наличие данных; символов процесса, указывающих на операции, которые следует выполнить над данными; линейных символов, отображающих поток между процессами и данными, а также инициации процессов; специальных символов, используемых для облегчения написания и чтения схемы.

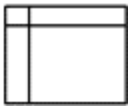
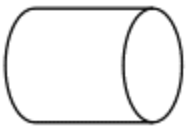
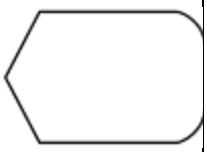
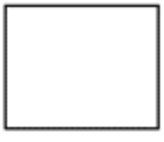
Схема ресурсов системы отображает конфигурацию блоков данных и обрабатывающих блоков, которая требуется для решения задачи или набора задач. Она состоит из символов данных, отображающих входные, выходные и запоминающие устройства вычислительной машины; символов процесса, отображающих процессоры; линейных символов, отображающих передачу данных между устройствами ввода-вывода и процессорами, а также передачу управления между процессорами; специальных символов, используемых для облегчения написания и чтения схемы.

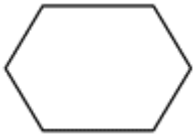
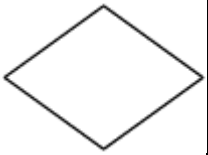
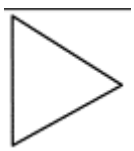
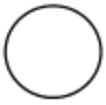
Операции обработки данных и носители информации изображаются на схеме соответствующими символами.

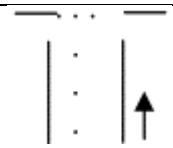
В ГОСТ 19.701-90 даются рекомендуемые формы символов и указывается, что не должны меняться углы и другие параметры, влияющие на соответствующую форму символов. Символы должны быть, по возможности, одного размера.

В табл. 4 приведены основные символы данных, используемые па схемах алгоритма.

Таблица 1. Основные символы данных, процессов, линий и специальные символы, используемые на схемах алгоритма

Название символа	Обозначение	Пояснение
Данные		Символ отображает данные; носитель данных не определен
Запоминаемые данные		Символ отображает хранимые данные в виде, пригодном для обработки; носитель данных не определен
Оперативное запоминающее устройство		Символ отображает данные, хранящиеся в оперативном запоминающем устройстве
Запоминающее устройство с прямым доступом		Символ отображает данные, хранящиеся в запоминающем устройстве с прямым доступом (магнитный диск, гибкий магнитный диск и т. и.)
Документ		Символ отображает данные, представленные на носителе в удобочитаемой форме (распечатка принтера, документ для оптического или магнитного считывания и т. п.)
Ручной ввод		Символ отображает данные, вводимые вручную во время обработки с устройств любого типа (клавиатура, переключатели, кнопки, световое перо и т. п.)
Дисплей		Символ отображает данные, представленные в удобочитаемой форме на носителе в виде отображающего устройства (экран для визуального наблюдения, индикаторы ввода информации и т. п.)
Процесс		Символ отображает функцию обработки данных любого вида (выполнение определенной операции или группы операций, приводящее к изменению значения, формы или размещения информации)

Название символа	Обозначение	Пояснение
Предопределенный процесс		Символ отображает предопределенный процесс, состоящий из одной или нескольких операций (шагов) программы, которые определены в под-программе, модуле и т. п.
Ручная операция		Символ отображает любой процесс, выполняемый человеком
Подготовка		Символ отображает модификацию команды (группы команд) для воздействия на последующую функцию (установка переключателя, модификация индексного регистра или инициализация программы)
Решение		Символ отображает решение или функцию переключательного типа, имеющую один вход и ряд альтернативных выходов, один и только один из которых может быть активизирован после вычисления условий, определенных внутри этого символа
Параллельные действия		Символ отображает синхронизацию двух или более параллельных операций
Граница цикла		Символ, состоящий из двух частей, отображает начало и конец цикла. Обе части символа имеют один и тот же идентификатор
Линия		Символ отображает поток данных или управления
Передача управления		Символ отображает непосредственную передачу управления от одного процесса к другому. Тип передачи управления должен быть назван внутри символа (запрос, вызов, событие)
Пунктирная линия		Символ отображает альтернативную связь между двумя или более символами. Кроме того, символ используют для обведения выбранного участка
Соединитель		Символ отображает выход в часть схемы и вход из другой части этой схемы и используется для обрыва линии и продолжения ее в другом месте

Название символа	Обозначение	Пояснение
Терминатор		Символ отображает выход во внешнюю среду и вход из внешней среды (начало или конец схемы программы)
Комментарий		Символ используют для добавления комментариев. Пунктирные линии в символе комментария связаны с соответствующим символом или группой символов. Текст комментариев должен быть помещен около ограничивающей скобки
Пропуск		Символ используют в схемах для отображения пропуска символа или группы символов

Примеры применения некоторых символов линий и специальных символов приведены на рис. 12 а, б.

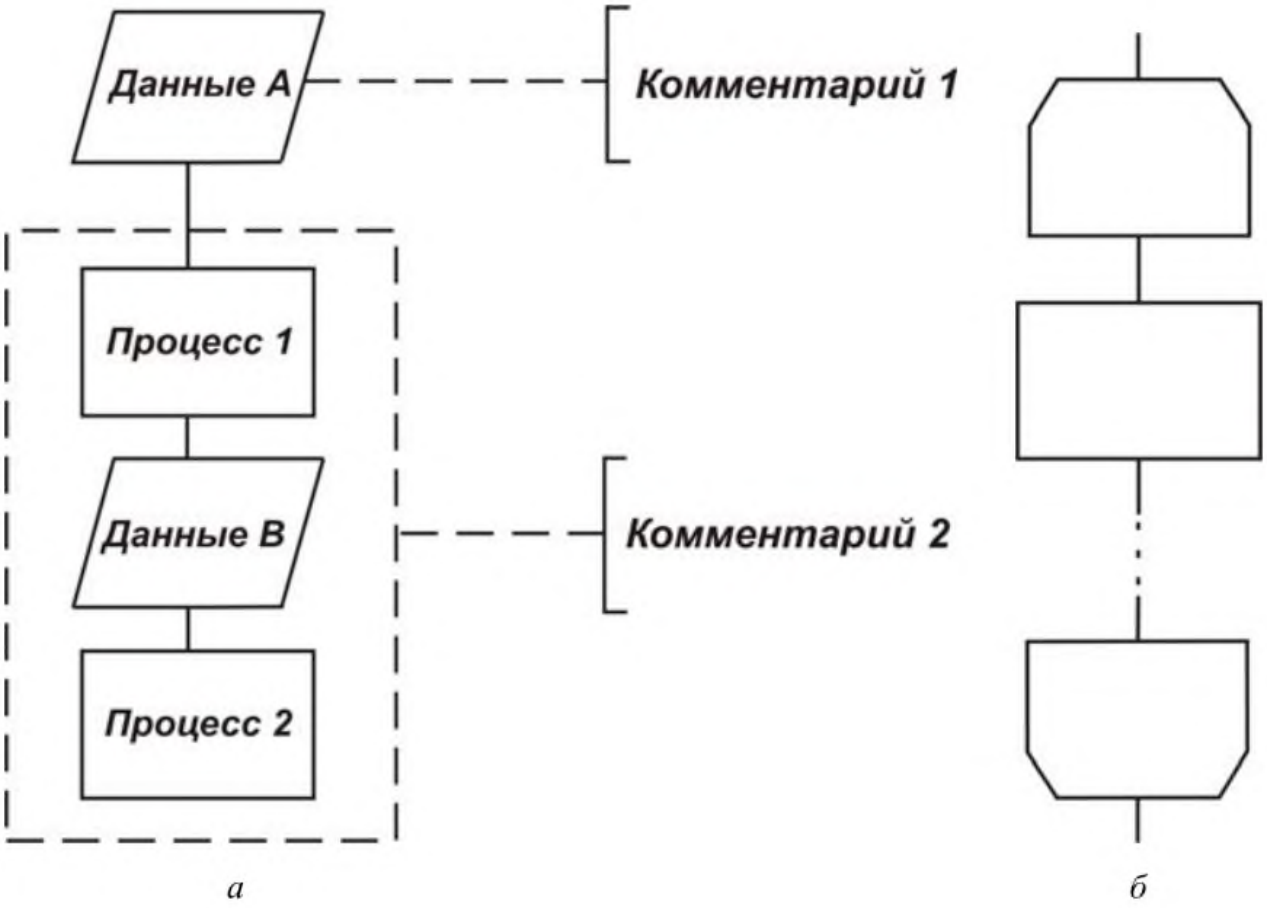


Рисунок 1. Примеры применения некоторых символов линий и специальных символов

Символы в схеме должны быть расположены равномерно. Следует придерживаться разумной длины соединений и минимального числа длинных линий.

В пределах одной схемы рекомендуется изображать блоки одинаковых размеров. При возникновении необходимости описать схему алгоритма блоки нумеруются.

Символы могут быть вычерчены в любой ориентации, но, по возможности, предпочтительной является горизонтальная ориентация. Зеркальное изображение формы символа обозначает одну и ту же функцию, но не является предпочтительным.

Минимальное количество текста, необходимого для понимания функции данного символа, следует помещать внутри данного символа слева направо и сверху вниз независимо от направления потока. Если объем текста, помещаемого внутри символа, превышает его размеры, следует использовать символ комментария.

В схемах слева над символом может использоваться идентификатор символов, который определяет символ при использовании его в других элементах документации (например, в листинге программы).

В схемах может использоваться описание символов – любая другая информация, например, для отображения специального применения символа с перекрестной ссылкой или для улучшения понимания функции как части схемы. Описание символа должно быть расположено справа над символом.

Потоки данных или потоки управления в схемах показываются линиями. Их стандартное направление - слева направо и сверху вниз.

В случаях, когда необходимо внести большую ясность в схему, на линиях используются стрелки. Если поток имеет направление, отличное от стандартного, то это направление должно указываться стрелками.

В схемах следует избегать пересечения линий. Пересекающиеся линии не имеют логической связи между собой, поэтому изменения направления в точках пересечения не допускаются.

Две или более входящие линии могут объединяться в одну исходящую линию. Если две или более линии объединяются в одну линию, место объединения должно быть смещено (рис. 13).

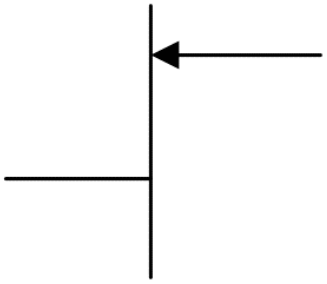


Рисунок 2. Место объединения двух линий

Линии в схемах должны подходить к символу либо слева, либо сверху, а исходить либо справа, либо снизу. Линии должны быть направлены к центру символа.

При необходимости линии в схемах следует разрывать во избежание излишних пересечений или слишком длинных линий, а также если схема состоит из нескольких страниц. Соединитель в начале разрыва называется внешним, а соединитель в конце разрыва - внутренним.

Ссылки к страницам могут быть приведены совместно с символом комментария для их соединителей (рис. 14).

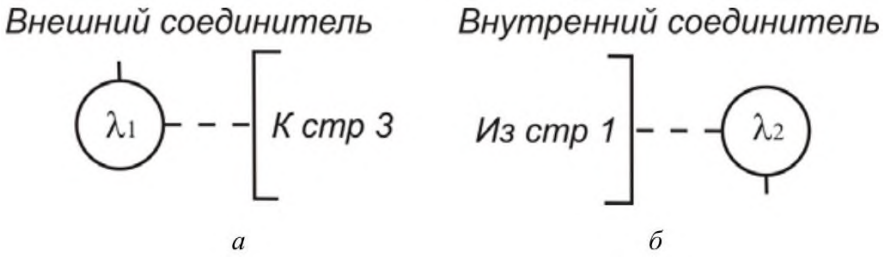


Рисунок 3. Ссылки к страницам

Несколько выходов из символа следует показывать:

1. несколькими линиями от данного символа к другим символам (рис. 15, а);

2. одной линией от данного символа, которая затем разветвляется в соответствующее число линий (рис. 15, а).

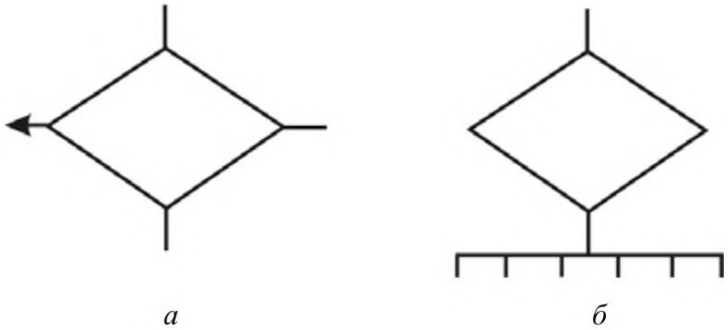


Рисунок 4. Несколько выходов из символа

Каждый выход из символа должен сопровождаться соответствующими значениями условий, чтобы показать логический путь, который он представляет, с тем чтобы эти условия и соответствующие ссылки были идентифицированы.

Вместо одного символа с соответствующим текстом могут быть использованы несколько символов с перекрытием изображения от переднего (первого) к заднему (последнему), каждый из которых содержит описательный текст (использование или формирование нескольких носителей данных или файлов, производство множества копий печатных отчетов или форматов перфокарт). При этом линии могут входить или исходить из любой точки перекрытых символов.

В табл. 5 помечено знаком «+», в каких схемах алгоритмов разрешено использовать некоторые наиболее применяемые символы.

Таблица 2. Использование наиболее применяемых символов

Наименование символа	Схема данных	Схема прог-раммы	Схема работы системы	Схема взаимодействия программ	Схема ресурсов системы
Данные	+	+	+	+	+
Документ, ручной ввод, дисплей	+	–	+	+	+
Процесс	+	+	+	+	+
Предопределенный процесс	–	+	+	+	–
Ручная операция	+	–	+	+	–
Подготовка	+	+	+	+	–
Решение	–	+	+	–	–
Параллельные действия	–	+	+	+	–
Граница цикла	–	+	+	–	–
Линия, пунктирная линия, соединитель, комментарии, пропуск	+	+	+	+	+
Терминатор	+	+	+	–	–

1.2. Основные алгоритмические структуры

Число реализованных конструкций конечно в любом языке программирования [23]. Структурной элементарной единицей алгоритма является команда, обозначающая один элементарный шаг обработки или отображения информации. Простая команда на языке блок-схем изображается в виде функционального блока «процесс», который имеет один вход и один выход. Из команд проверки условий и простых команд образуются составные команды, имеющие более сложную структуру, но тоже один вход и один выход.

Алгоритм любой сложности может быть представлен комбинацией трех базовых структур: следование; ветвление (в полной и сокращенной форме); цикл (с предусловием или постусловием).

Характерной особенностью этих структур является наличие у них одного входа и одного выхода.

I. Линейные алгоритмы

Базовая структура «следование» означает, что не сколько операторов выполняются последовательно друг за другом, и только один раз за время выполнения программы. Структура «следование» используется для реализации задач, имеющих линейный алгоритм решения. Это означает, что такой алгоритм не

содержит проверок условий и повторений, действия в нем выполняются последовательно, одно за другим.

Линейная структура является основной [24]. Она означает, что действия выполняются друг за другом (рис. 16).

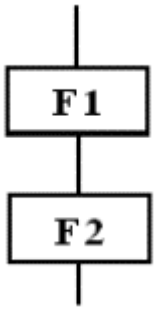


Рисунок 5. Линейная структура

Прямоугольник, показанный на рисунке, может представлять как одну единственную команду, так и множество операторов, необходимых для выполнения сложной обработки данных, где F1 и F2 – некоторые команды для соответствующего исполнителя. Команды записываются с помощью операции присваивания.

Присваивание переменной какого-либо значения или присваивание одной переменной значения другой переменной является наиболее часто выполняемым действием в программе, написанной на любом языке программирования. В языке программирования присваивание является операцией. Это означает, что при выполнении этой операции происходит не только присваивание значения определенной переменной, но и возвращается некоторое значение.

II. Разветвляющиеся алгоритмы

Второй базовой структурой является «ветвление» [23]. Эта структура обеспечивает, в зависимости от результата проверки условия, выбор одного из альтернативных путей работы алгоритма, причем каждый из путей ведет к общему выходу, так что работа алгоритма будет продолжаться независимо от того, какой путь будет выбран.

Существует структура с полным и неполным ветвлением.

Структура с полным ветвлением (*если – то – иначе*) записывается так:

Если< условие >

то <действия 1>

иначе <действия 2>

Все если

Команда выполняется так: если <*условие*> является истинным, то выполняются <*действия 1*>, записанные после ключевого слова *то*, если <*условие*> является ложным, то выполняются <*действия 2*>, записанные после слова *иначе* (рис. 17).

Структура с неполным ветвлением (*если – то*) не содержит части, начинающейся со слова *иначе*:

Если <условие>

то <действия 1>

Все если

Команда выполняется так: если *<условие>* является истинным, то выполняются *<действия 1>*, записанные после ключевого слова *то*.

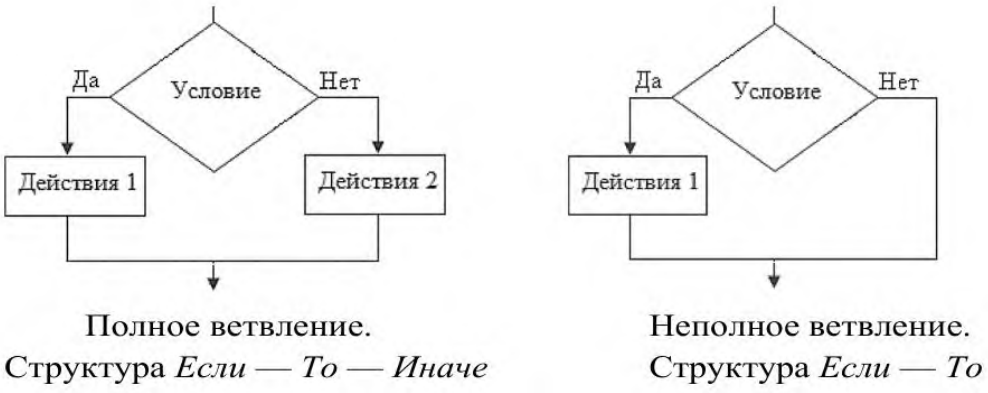


Рисунок 6. Разветвляющиеся алгоритмы

III. Циклические алгоритмы

При составлении алгоритмов решения большинства задач возникает необходимость в неоднократном повторении одних и тех же команд. Алгоритм, составленный с использованием многократных повторений одних и тех же действий (циклов), называется циклическим. Однако слово «неоднократно» не означает «до бесконечности». Организация циклов, никогда не приводящая к остановке в выполнении алгоритма («зацикливание» алгоритма), нарушает требование его результативности получения результата за конечное число шагов.

Блок, для выполнения которого организуется цикл, называется телом цикла (рис. 18). Остальные операторы служат для управления процессом повторения вычислений: это начальные установки, проверка условия продолжения цикла и модификация параметра цикла. Один проход цикла называется итерацией.

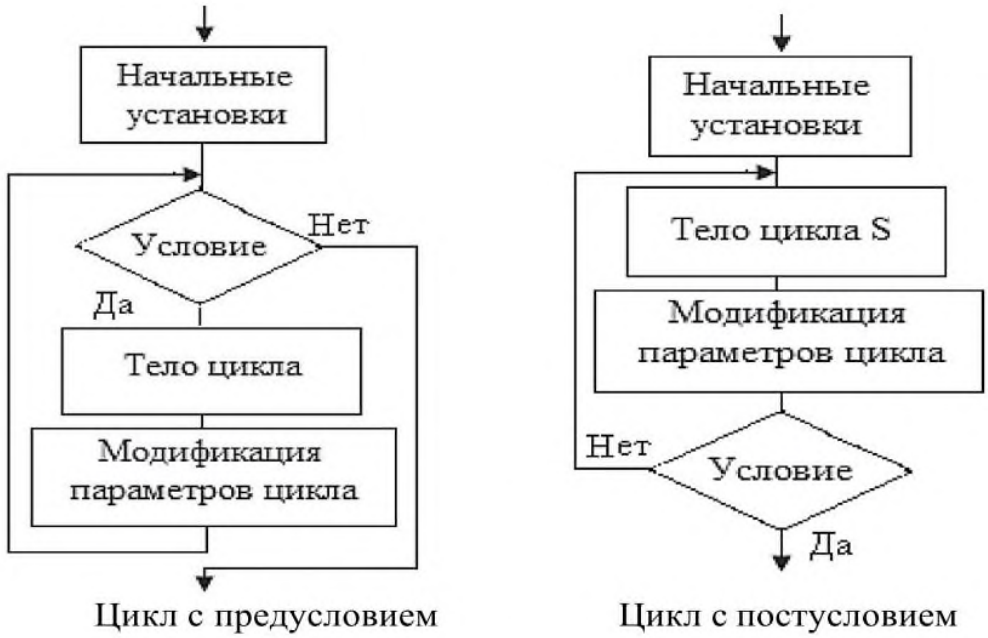


Рисунок 7. Циклические алгоритмы

Начальные установки служат для того, чтобы до входа в цикл задать значения переменных, которые в нем используются.

Проверка условия продолжения цикла выполняется на каждой итерации либо до тела цикла (цикл с предусловием), либо после тела цикла (цикл с постусловием). Тело цикла с постусловием всегда выполняется хотя бы один раз. Проверка необходимости выполнения цикла с предусловием делается до начала цикла, поэтому возможно, что он не выполнится ни разу.

При конструировании циклов следует соблюдать обязательное условие результативности алгоритма (т. е. его окончания за конечное число шагов). Практически это означает, что в условии должна быть переменная, значение которой изменяется в теле цикла. Причем, изменяется таким образом, чтобы условие в конечном итоге перестало выполняться. Такая переменная называется управляющей переменной цикла или параметром цикла.

Еще один вид циклов – цикл с параметром, или арифметический цикл. Тело цикла выполняется, пока параметр цикла пробегает множество значений от начального до конечного. Параметр цикла определяет количество повторений тела цикла.

1.3. Примеры записи алгоритма с помощью языка блок-схем

Линейным принято называть вычислительный процесс, в котором операции выполняются последовательно, в порядке их записи [24]. Каждая операция является самостоятельной, независимой от каких-либо условий. На схеме блоки, отображающие эти операции, располагаются в линейной последовательности. Линейные вычислительные процессы имеют место, например, при вычислении арифметических выражений, когда имеются конкретные числовые данные и над ними выполняются соответствующие условию задачи действия.

На рис. 19 показан пример линейного алгоритма, определяющего процесс вычисления арифметического выражения $y = (b^2 - ac) : (a + c)$ для различных значений переменных a, b и c.

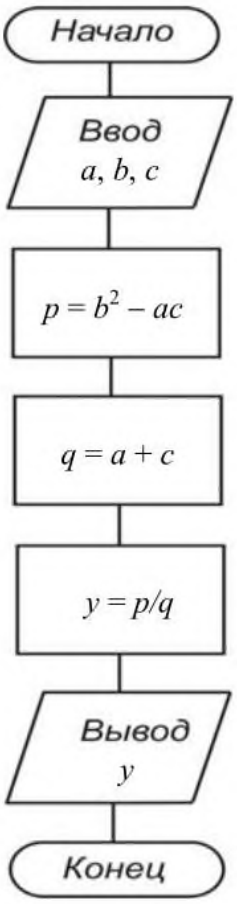


Рисунок 8. Пример линейного алгоритма

Вычислительный процесс называется разветвляющимся, если для его реализации предусмотрено несколько направлений (ветвей). Каждое конкретное направление процесса обработки данных является отдельной ветвью вычислений. Ветвление в программе – это выбор одной из нескольких последовательностей команд при выполнении программы. Выбор направления зависит от заранее определенного признака, который может относиться к исходным данным, к промежуточным или конечным результатам. Признак характеризует свойство данных и имеет два или более значений.

Разветвляющийся процесс, включающий в себя две ветви, называется простым, более двух ветвей – сложным. Сложный разветвляющийся процесс можно представить с помощью комбинации простых разветвляющихся процессов. Направление ветвления выбирается логической проверкой, в результате которой возможны два ответа: «да» («+») - условие выполнено и «нет» («-») условие не выполнено.

Следует иметь в виду, что, хотя на схеме алгоритма должны быть показаны все возможные направления вычислений в зависимости от выполнения определенного условия (или условий), при однократном прохождении программы процесс реализуется только по одной ветви, а остальные исключаются. Любая ветвь, по которой осуществляются вычисления, должна приводить к завершению вычислительного процесса.

На рис. 20 показан пример алгоритма с разветвлением для вычисления следующего выражения:

$$y = \begin{cases} a + b, x \leq 0, \\ \dfrac{c}{b}, x > 0. \end{cases}$$

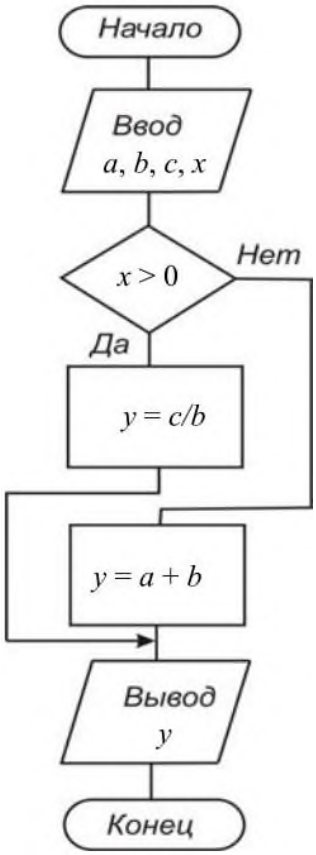


Рисунок 9. Пример алгоритма с разветвлением

В зависимости от истинности или ложности условия, указанного в символе решение, значение функции вычисляется по одному из двух альтернативных выражений.

На рис. 21 показан еще один пример алгоритма с разветвлением для вычисления значения ступенчатой функции:

$$z = \begin{cases} 0{,}5x^2 + 1, x \leq a, \\ \cos x\,, a < x < b, \\ \operatorname{tg}^2 x, x \geq b. \end{cases}$$

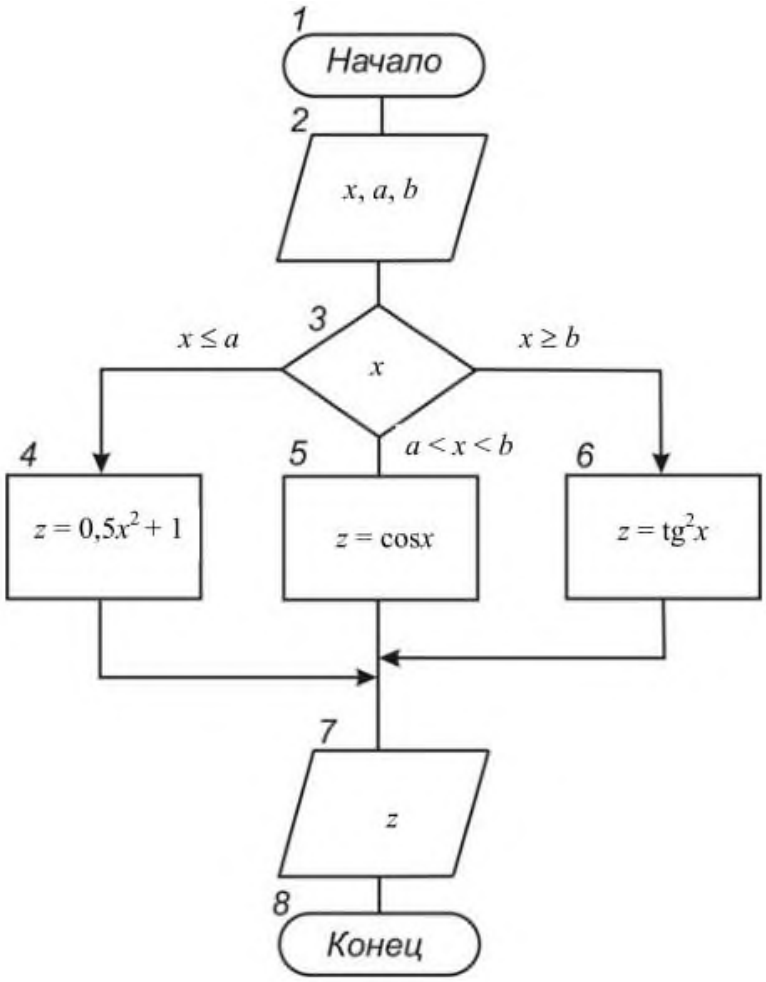


Рисунок 10. Пример алгоритма с разветвлением для вычисления значения ступенчатой функции

В алгоритме после ввода любых значений x, a и b и проверки условия в символе решение 3 значение функции z может вычисляться по одной из трех альтернативных ветвей: в символе процесс 4, 5 или 6. При этом, так как количество альтернативных ветвей больше двух, около соответствующею вывода символа решение 3 указывается условие перехода на соответствующую ветвь, а внутри самого символа решение указывается лишь имя переменной x.

Результаты вычисления значения функции по любой из альтернативных ветвей выводятся в символе ввод-вывод 7.

На рис. 22 приведен алгоритм, определяющий принадлежность точки с координатами (X, Y) кругу радиуса R.

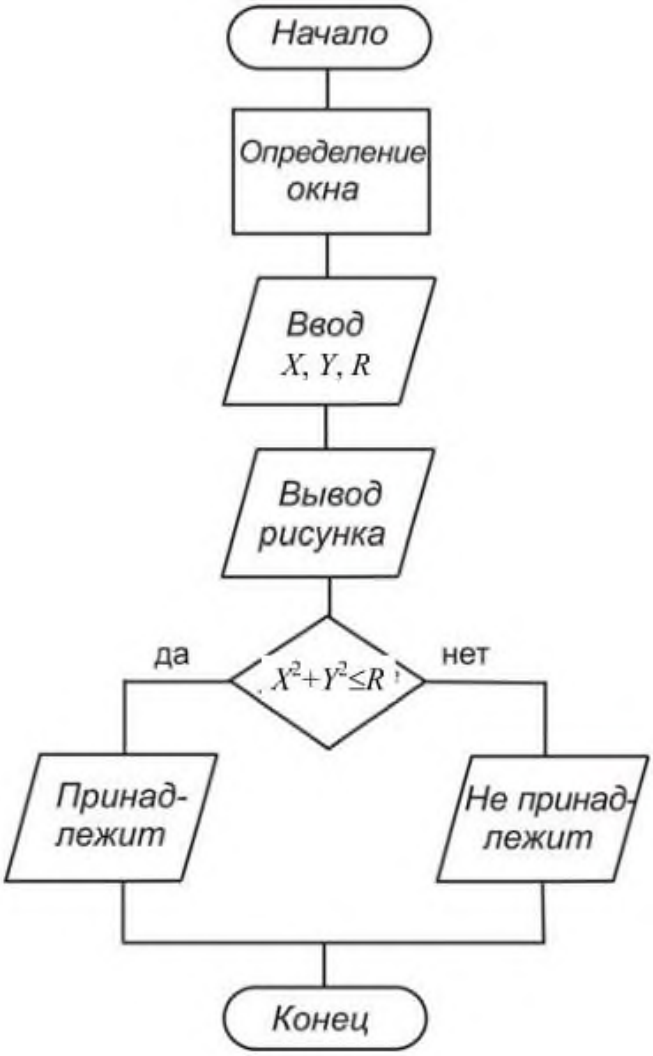


Рисунок 11. Алгоритм, определяющий принадлежность точки кругу

Сначала в символе процесса Определение окна задаются размеры окна, в котором будет выводиться рисунок, например: (-100,-100) - (100, 100).

Алгоритм предусматривает ввод любых значений X, Y, R вывод рис. 23, на котором показаны оси, круг с радиусом R и точка с координатами (X, Y).

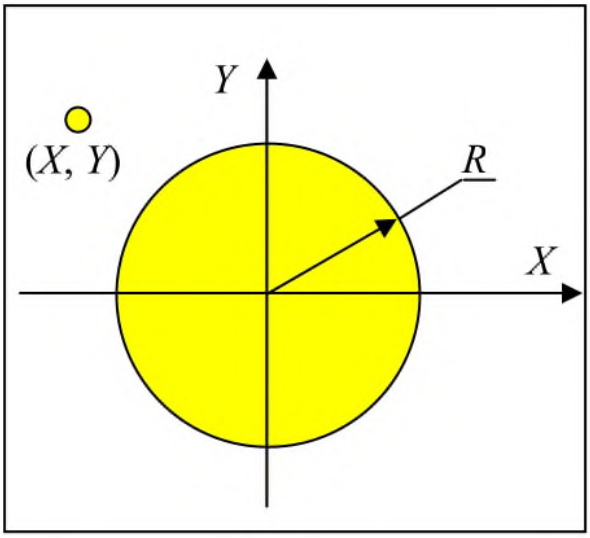


Рисунок 12. Пример вывода рисунка алгоритма, определяющего принадлежность точки кругу

Так как уравнение окружности: $X^2 + Y^2 = R^2$, то точка с координатами (X, Y) принадлежит заданной области (кругу), если выполняется условие

$$X^2 + Y^2 = R^2$$

В результате выполнения алгоритма в случае, если точка принадлежит кругу, выводится сообщение «Принадлежит», иначе - «Не принадлежит».

Среди разветвляющихся алгоритмических процессов в отдельную группу можно выделить переключательные.

Переключательные алгоритмические процессы позволяют выбрать один вариант из нескольких возможных вариантов. Их можно представить с помощью простых разветвляющихся процессов, но можно (что чаще проще) и с помощью структуры «выбор». Эта структура имеет вид, представленный на рис. 24.

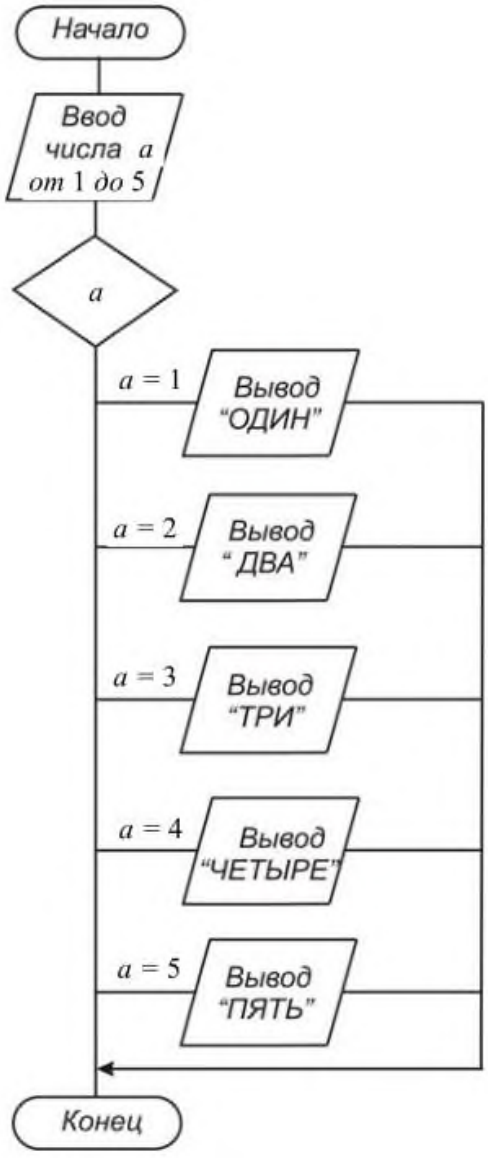


Рисунок 13. Пример алгоритма со структурой «выбор»

На рисунке приведен переключательный алгоритм, выдающий словесное значение числа a. В данном примере введенное число сравнивается с числами от одного до пяти, и если a равно одному из этих чисел, то па экран выводится словесное значение числа.

Каждый выход из символа решение должен сопровождаться соответствующими значениями условий, чтобы показать логический путь, который он представляет, с тем чтобы эти условия и соответствующие ссылки были идентифицированы.

Выход из любой альтернативной ветви алгоритма приводит к его завершению, при этом остальные условия не проверяются.

Циклическими называются алгоритмы, содержащие циклы. Цикл – это многократно повторяемый участок алгоритма.

В организации цикла можно выделить следующие этапы:

- подготовка (инициализация) цикла (И);
- выполнение вычислений цикла (тело цикла) (Т);
- модификация параметров (М);
- проверка условия окончания цикла (У);
- продолжение (П).

На рис. 25 показана последовательность этих этапов.

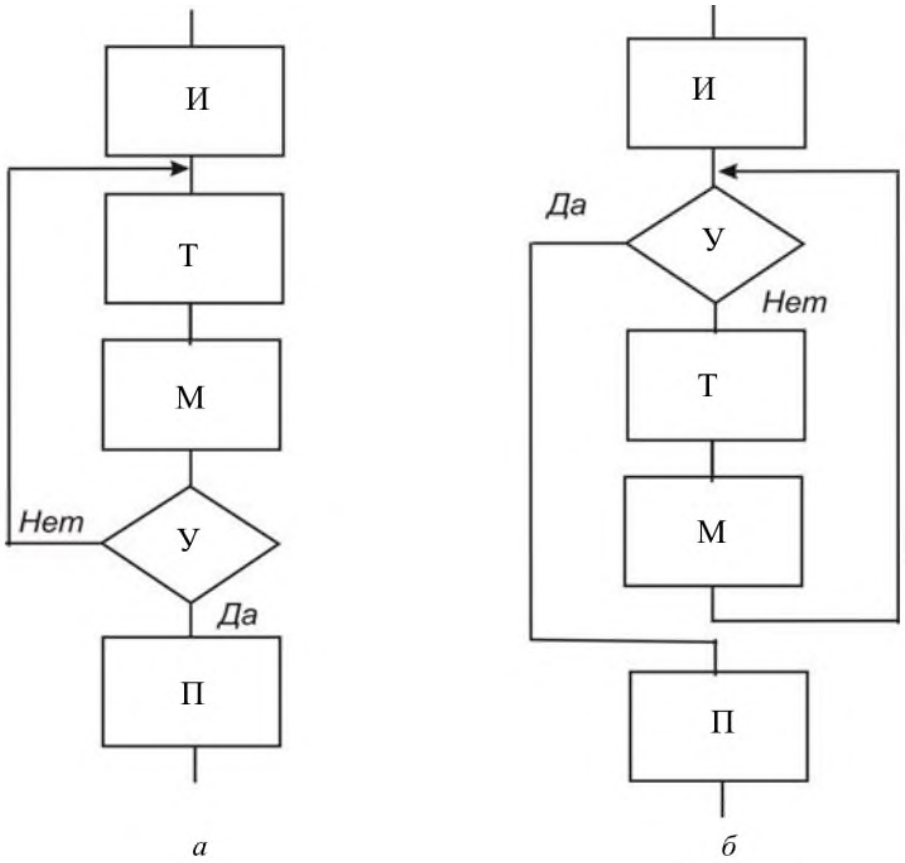


Рисунок 14. Последовательность этапов цикла

На этапе инициализации присваиваются начальные значения переменной цикла x тем самым обеспечивается выполнение тела цикла на первом цикле.

Тело цикла в общем случае может содержать любое количество операций (в частности, может вообще отсутствовать).

На этапе модификации параметров в случае, если существует закономерность изменения значений переменной цикла, а также если эти значения используются

строго последовательно, то к текущему значению переменной цикла прибавляется шаг Δx учетом знака

$$x = x + \Delta x.$$

Если не существует никакой закономерности изменения значений переменной цикла, то эти значения могут храниться в одномерном массиве в виде $x(i)$. В этом случае в качестве переменной цикла будет выступать i, модификация которой будет также производиться путем изменения се значения па единицу

$$i = i + 1.$$

Проверка условия окончания цикла чаще всего предполагает сравнение текущего значения переменной цикла с е конечным значением. В общем случае условие окончания цикла может быть явно не есвязано ни со значением переменной цикла, ни с ее конечным значением.

Порядок выполнения этапов, например, Т и М, может изменяться. В зависимости от расположения проверки условия окончания цикла различают циклы с нижним и верхним окончаниями. Для цикла с нижним окончанием (рис. 25, а) тело цикла выполняется как минимум один раз, так как сначала производятся вычисления, а затем проверяется условие выхода из цикла. В случае цикла с верхним окончанием (рис. 25, б) тело цикла может не выполниться ни разу в случае, если сразу соблюдается условие выхода.

После выхода из цикла алгоритм переходит на этап продолжения программы (этап П на рис. 25).

На рис. 26 показан пример циклического алгоритма вычисления суммы десяти чисел. Здесь в качестве переменной цикла используется переменная i с начальным значением, равным единице, и конечным значением, равным 10, и шагом, равным единице.



Рисунок 15. Пример циклического алгоритма вычисления суммы десяти чисел

В этом цикле проверка условия выхода из цикла выполняется в конце цикла. При этом тело цикла повторится десять раз.

Для удобства на схеме алгоритма с помощью символов комментарий поясняется, какие действия выполняются на каждом шаге алгоритма.

Цикл называется детерминированным, если число повторений тела цикла заранее известно или может быть определено из выражений. Цикл на рис. 27, а предписывает выполнять тело цикла для всех значений переменной цикла *X* в заданном диапазоне от 0 до 20 с шагом 2.

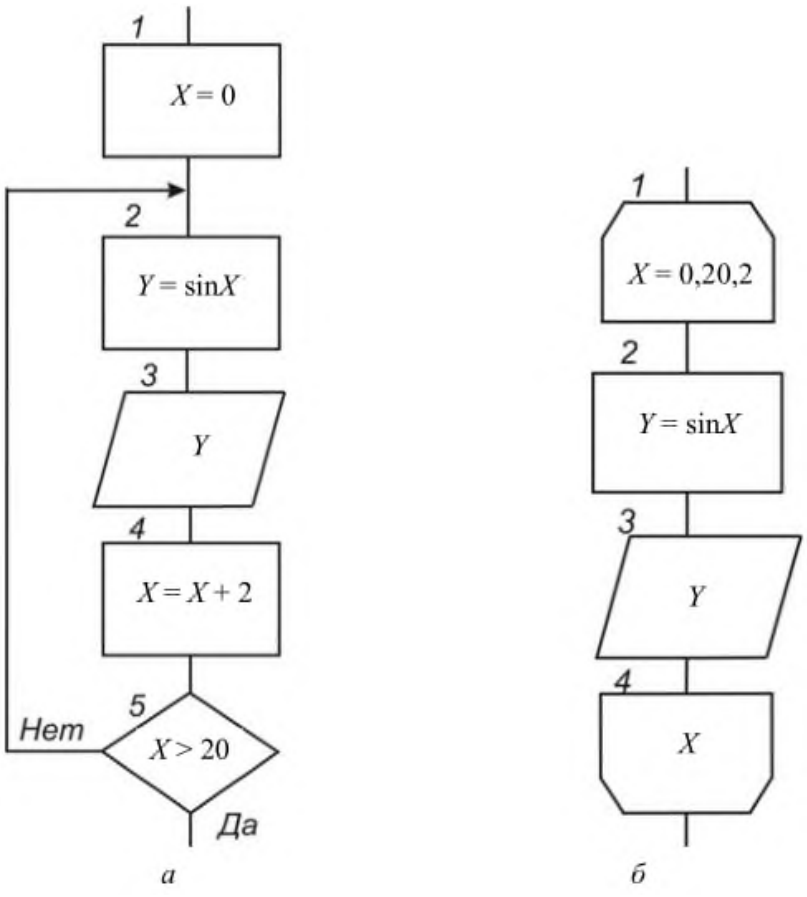


Рисунок 16. Пример циклического детерминированного алгоритма

Инициализация выполняется в символе 1, а к телу цикла относятся символы 2 и 3 на схеме алгоритма. Модификация параметров выполняется в символе 4, а проверка условия окончания цикла – в символе 5.

Число повторений цикла, определяемое как отношение разности конечного (20) и начального (0) значений переменной цикла (X) к шагу (2) плюс один, в данном примере равно 11.

На рис. 27, б приведен другой вариант реализации этого же цикла с применением специальных символов начала и конца цикла 1 и 4. В символе 1 задаются начальные и конечные значения переменной цикла X а также шаг изменения X. К телу цикла относятся символы 2 и 3 на рис. 27, б. Модификация параметров цикла и проверка условия окончания цикла выполняются в символе 4.

Вычисление суммы и произведения нескольких чисел, а также подсчет количества в большинстве языков программирования выполняется постепенно, путем накапливания.

1. Сумма вычисляется по формуле: сумма = сумма + слагаемое.

Перед вычислениями начальному значению суммы нужно присвоить значение «ноль».

2. Произведение вычисляется по формуле: произведение = произведение × сомножитель.

Перед вычислениями начальному значению произведения нужно присвоить значение единицы.

3. Подсчет количества выполняется по формуле, подобной формуле для вычисления суммы: количество = количество + 1.

Начальному значению количества присваивается значение «ноль».

Алгоритмы с накапливанием часто применяются при расчете конечных рядов и произведений. Па рис. 28 приводятся алгоритмы вычисления конечного ряда $\sum_{i=2}^{100} \frac{i+1}{i+2}$ и конечного произведения $\prod_{i=1}^{25} \frac{i^2}{i^2+2i+3}$ соответственно.

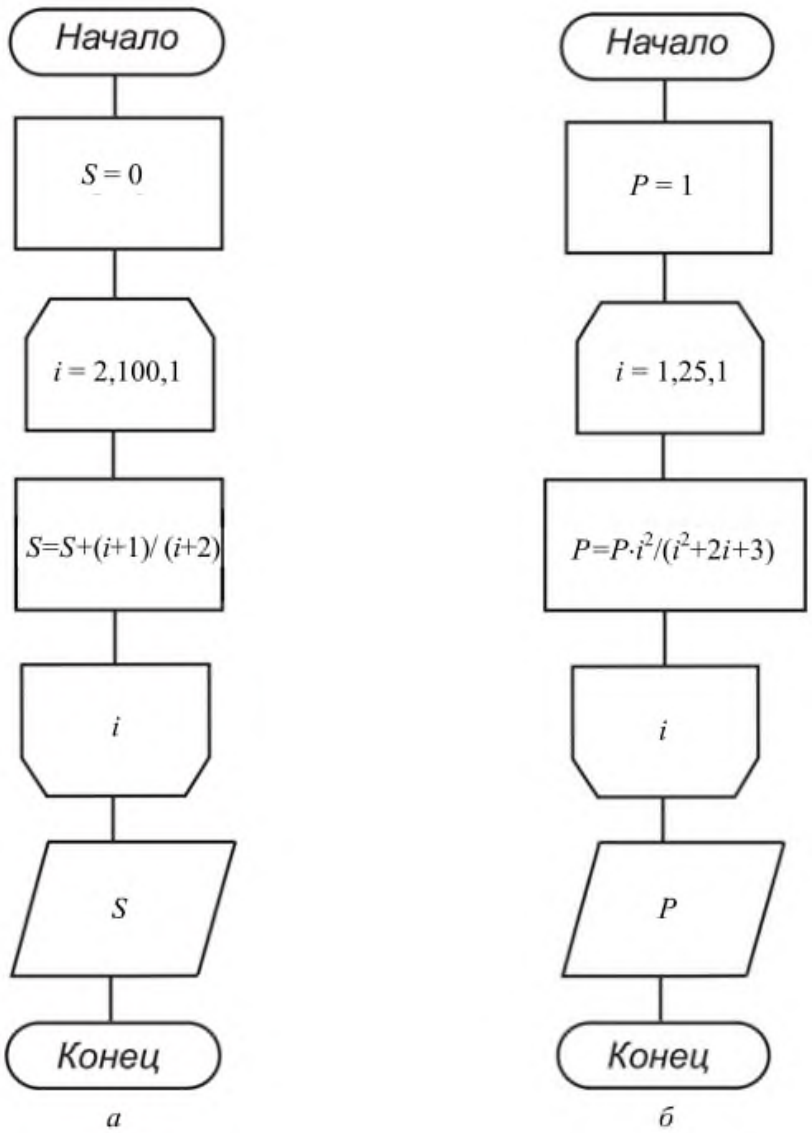


Рисунок 17. Пример алгоритмов с накоплением

Массив – это совокупность элементов, имеющих одинаковое имя и отличающихся номером (индексом) их расположения в последовательности.

Одномерный массив - последовательность чисел $A(1), A(2), \dots, A(n)$. Размерность массива равна количеству элементов в нем.

Дан одномерный массив $A(i)$ длиной 20, упорядоченный по убыванию. Переупорядочить массив $A(i)$ по возрастанию его элементов, используя первоначальную упорядоченность массива, и сохранить его значения в массиве $B(i)$. Вывести полученный массив $B(i)$ на дисплей.

Сначала рассмотрим словесный алгоритм решения задачи:

1. В цикле последовательно перебрать все элементы массива $B(i)$ (начиная с первого по двадцатый).

2. Присвоить им значения соответствующих элементов массива $A(21-i)$ (начиная с конца).

3. Вывести полученное значение на дисплей

4. Повторять тело цикла 20 раз.

На рис. 29 приводится соответствующая схема алгоритма.

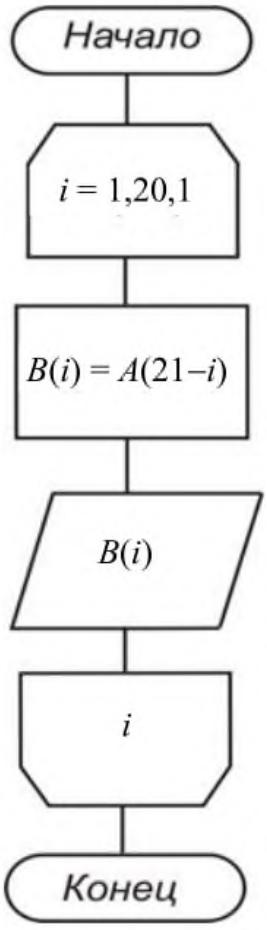


Рисунок 18. Алгоритм переупорядочивания массива

Символы, образующие текст, можно рассматривать как элементы одномерного массива. При этом номер символа в тексте совпадает с индексом элемента в массиве. В большинстве языков программирования для работы с текстом используют строковые константы, доступ к символам которых организуется с помощью специальных функций по номеру символа в константе аналогично, как и к элементам массива.

Дано слово, хранящееся в массиве $A(N)$. Определить, есть ли в этом слове символ «а». Если есть, то каков номер первого символа «а» в слове.

На рис. 30 приводится соответствующий алгоритм. Определив количество символов в слове, в цикле организуем их последовательное сравнение с символом «а». Если такой символ встретится, то выйдем из цикла и выведем

сообщение «Символ «а» хранится на i-м месте». Иначе после проверки всех символов в слове выведем сообщение «Нет символа «а»».

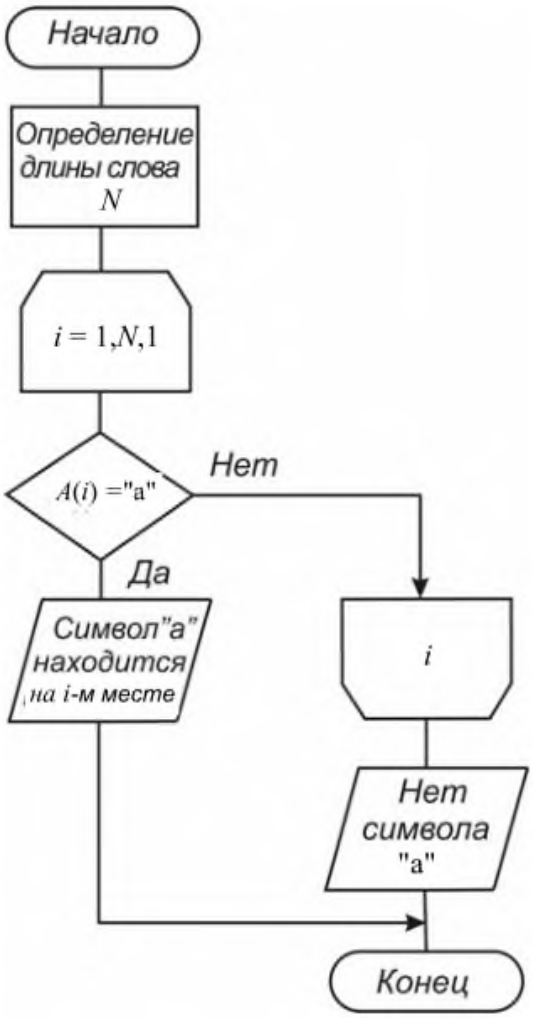


Рисунок 19. Алгоритм определения наличия символа в слове

Возможны случаи, когда внутри тела цикла необходимо повторять некоторую последовательность операторов, т. е. организовывать внутренний цикл. Такая структура получила название цикла в цикле, или вложенных циклов. Вложенные циклы не могут пересекаться: пока не закончился внутренний цикл, внешний не может завершиться. Глубина вложения циклов, т. е. количество вложенных друг в друга циклов, может быть различной. При использовании такой структуры для экономии машинного времени необходимо выносить из внутреннего цикла во внешний все операторы, которые не зависят от параметра внутреннего цикла.

Если число повторений вложенного цикла известно или зависит от значения переменной, модифицируемой в цикле, то в схеме алгоритма можно использовать символ границы цикла.

Рассмотрим фрагмент алгоритма, вычисляющего значения функции $A = \prod_{k=1}^{n} \sqrt{x^2 + k^2}$ при x, изменяющемся от 0 до 5 с шагом 0,5 для любого значения n.

Вычисление произведения организуется в теле цикла путем накапливания - последовательного перебора и умножения отдельных сомножителей $A = A \cdot f(x,k)$ (рис. 31). При этом первое произведение получается путем умножения первого сомножителя на единицу ($A = 1$). Если необходимо организовать вычисление суммы, то в теле цикла нужно последовательно перебирать и складывать отдельные слагаемые по формуле $A = A + f(x,k)$, а первая сумма получится путем сложения первого слагаемого с нулем, т. с. перед внутренним циклом необходимо присвоить A значение, равное нулю.

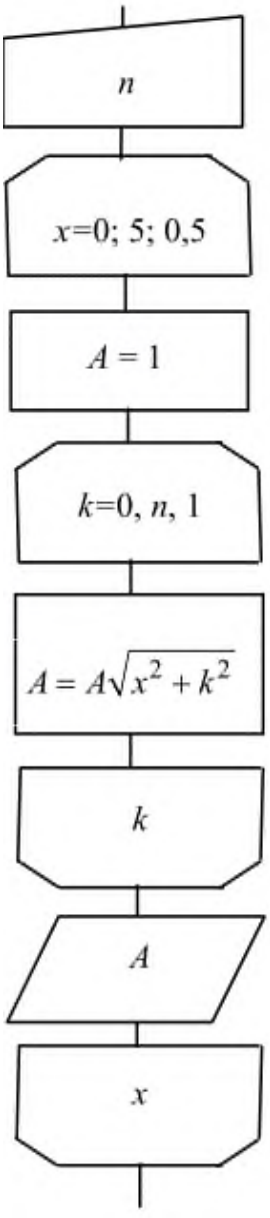


Рисунок 20. Алгоритм вычисления произведения

С помощью вложенных циклов выполняется и задача сортировки. Сортировка – это расположение чисел в порядке возрастания или убывания.

Наиболее распространенный и простой метод сортировки – метод «пузырька». Он требует минимального объема памяти для данных, но затраты времени на этот метод велики. Суть метода «пузырька» в следующем. Пусть дано n чисел и

необходимо расположить их (для определенности) в порядке возрастания. При упорядочении можно использовать следующий алгоритм:

1. числа сравниваются попарно: первое со вторым; второе с третьим; ..., i-е с $(i + 1)$-м;

2. если меньшее стоит в паре на втором месте, то числа меняются местами.

За один такой просмотр массива минимальное число перемещается по крайней мере на одно место вверх (вперед), а максимальное – в самый конец, т. е. минимальное число как легкий пузырек воздуха в жидкости постепенно «всплывает» в начало последовательности. Отсюда название метода.

За $(n - 1)$ просмотров произойдет полное упорядочение массива при любом исходном расположении чисел в нем.

Особенностью итерационного цикла является то, что число повторений операторов тела цикла заранее неизвестно, а зависит от значений параметров (некоторых переменных), участвующих в вычислениях. Для его организации используется цикл типа «пока». Выход из итерационного цикла осуществляется в случае выполнения заданного условия.

На каждом шаге вычислений происходят последовательное приближение и проверка условия достижения искомого результата.

Алгоритм, в состав которого входит итерационный цикл, называется итерационным алгоритмом. Итерационные алгоритмы используются при реализации итерационных численных методов. В итерационных алгоритмах необходимо обеспечить обязательное достижение условия выхода из цикла (сходимость итерационного процесса). В противном случае произойдет зацикливание алгоритма, т. е. не будет выполняться основное свойство алгоритма - результативность (или конечность). Это свойство состоит в том, что алгоритм должен приводить к решению задачи за конечное число шагов.

Рассмотрим применение итерационного метода для приближенного вычисления функций.

Вычислить значение функции $\sin x$ по приближенной формуле

$$f = \sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$$

с погрешностью $d = 10^{-5}$.

Функция f представляет собой сумму членов степенного ряда:

$$f = u_1 + u_2 + \cdots + u_n.$$

Условие окончания вычисления функции

$$|u_n - u_{n+1}| \le d,$$

где u_n – общий член ряда; u_{n+1} – следующее слагаемое.

Слагаемые можно определить:

1. непосредственно по формуле (но при большом значении п это неэффективно);

2. используя предыдущее значение.

Запишем формулы для вычисления u_n учитывая, что в выражении для $\sin x$ присутствуют только нечетные степени x и факториалы нечетных чисел. Если общий член ряда равен

$$u_n = \frac{x^{2n-1}}{(2n-1)!},$$

то следующее слагаемое

$$u_{n+1} = \frac{x^{2n+1}}{(2n+1)!}.$$

Откуда u_{n+1} через u_n можно определить как

$$u_{n+1} = -\frac{x^2}{2n(2n+1)}u_n.$$

Рассмотрим сначала словесный алгоритм.

1. Ввести значения аргумента x и точность вычисления d.
2. Положить $f = 0, n = 1, u_{n+1} = x$.
3. Заменить $u_n = u_{n+1}$.
4. Вычислить $f = f + u_n$.
5. Увеличить $n = n + 2$.
6. Вычислить

$$u_{n+1} = -\frac{x^2}{2n(2n+1)}u_n.$$

7. Пока $|u_n - u_{n+1}| > d$, повторять пункты 3–6.
8. Вывести значение f.

Схема алгоритма для рассмотренного алгоритма приведена на рис. 32.

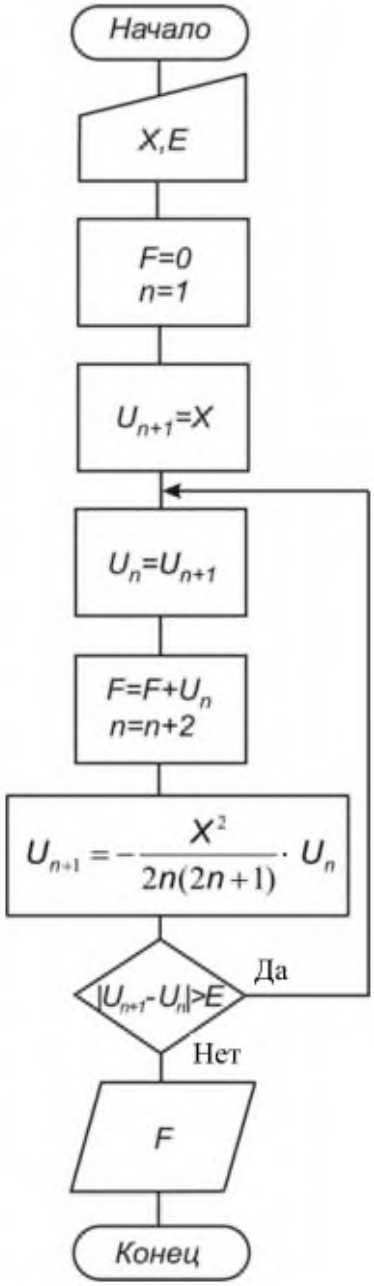


Рисунок 21. Алгоритм приближенного вычисления функции $\sin x$

1.4. Подпрограммы

При разработке программы имеют место случаи, когда один и тот же набор операторов, реализующих определенную цель, необходимо повторить многократно в нескольких местах программы [25]. Чтобы избавиться от дублирования и тем самым упростить процесс написания и отладки программы, повторяющиеся фрагменты выделяют в отдельные модули, называемые подпрограммами. Подпрограмма - поименованная логически законченная группа операторов языка. Упоминание имени подпрограммы в тексте программы приводит к ее активизации и называется вызовом подпрограммы. Существуют два вида подпрограмм – процедуры и функции.

Процедура – это независимая поименованная часть программы, вызываемая по имени, для выполнения определенных действий. Структура процедуры аналогична структуре программы. Процедура не может выступать как операнд в выражении.

Функция – это независимая поименованная часть программы, вызываемая по имени, для выполнения определенных действий. Однако в отличие от процедуры, функция передает в точку вызова скалярное значение, а имя функции должно входить в выражение как операнд.

Обычно все процедуры и функции языка подразделяются на две группы: встроенные и определенные пользователем. Встроенные (стандартные) процедуры и функции являются частью языка и могут быть вызваны по строго фиксированному имени без предварительного определения в разделе описания. Большой набор стандартных подпрограмм языка существенно облегчает работу программиста, однако иногда необходимы специфические действия, не предусмотренные стандартными процедурами и функциями. В этом случае программист может разработать свою подпрограмму. Ее называют пользовательской (определенной пользователем). Вызову пользовательской подпрограммы должно предшествовать ее описание.

Процедура пользователя представляет собой поименованную группу операторов, которые выполняются при каждом вызове процедуры по имени из любого места раздела операторов. Описание процедуры производится в разделе объявления подпрограмм в программе и включает описания заголовка и тела процедуры.

Функция пользователя возвращает в точку вызова функции искомое значение и представляет собой поименованную группу операторов, которые реализуют определенные действия, выполняемые при каждом вызове функции.

Тело подпрограммы состоит из директив, раздела описаний и раздела операторов. Раздел описаний подпрограммы может включать в себя описание констант, типов, переменных, подпрограмм низшего уровня. Все идентификаторы, которые были описаны в подпрограмме, называются локальными. Они недоступны в вызывающей их программе. Идентификаторы, описание которых выполнено в головной программе, называются глобальными. Такие глобальные данные доступны как в головной программе, так и в любом месте вызываемой подпрограммы. Если локальные идентификаторы совпадают с глобальными, то в подпрограмме глобальные данные недоступны.

При описании подпрограммы (процедуры или функции) за указанием ее имени может следовать необязательный список формальных параметров, заключенный в круглые скобки.

Параметры могут иметь любой тип, в том числе и структурированный. Однако необходимо внимательно следить за тем, чтобы типы фактического и соответствующего ему формального параметра совпадали в случае передачи параметра-переменной и были совместимы по присваиванию в случае передачи параметра–значения.

Подводя промежуточный итог, можно выделить основные правила написания подпрограмм’.

- необходимо определить круг задач, возлагаемых на проектируемую подпрограмму;

- в зависимости от местоположения точек вызова и решаемых задач следует определить, процедура эго будет или функция;

- необходимо определить, какие данные должны передаваться в подпрограмму в качестве параметров; это должны быть лишь те данные.

изменение которых в точке вызова делает подпрограмму универсальной, пригодной для широкого использования;

- следует определить, какие данные будут локальными, а какие целесообразнее оставить глобальными (глобальных данных должно быть как можно меньше);

- необходимо выбрать способ передачи данных (параметры-значения, параметры-константы, параметры-переменные, параметры без типа); от способа передачи данных зависят эффективность и правильность выполнения программы.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.